Cost Manager | Final Project Submission

--- TEAM INFO ---

Team Manager: Eliya Zakay

Team Members:

1. Polina Katsnelson | ID: 342854940 | Phone: 0587514036 | Email: katsnelsonp@gmail.com
2. Sam Sotil | ID: 207970807 | Phone: 0544500290 | Email: sotilsam@gmail.com
3. Eliya Zakay | ID: 323094847 | Phone: 0545874150 | Email: eliyazak3@gmail.com

Video Link: https://youtu.be/3IVD6tUVs6Y

Collaborative Tools Used:

We used GitHub for version control and task management,
and WhatsApp for team communication and coordination.

--- CODE FILES ---

The following pages contain all source code files for the project.

src/App.jsx

```jsx
// App.jsx
// Root component of the Cost Manager application.
// Manages navigation state and renders the active page.

import { useState, useMemo } from 'react';
import { Box, Container } from '@mui/material';
import NavBar from './components/NavBar';
import AddCostForm from './components/AddCostForm';
import MonthlyReport from './components/MonthlyReport';
import PieChartView from './components/PieChartView';
import BarChartView from './components/BarChartView';
import Settings from './components/Settings';
import { openCostsDB } from './db';

// Constants for the database used throughout the application
const DB_NAME = 'costsdb';
const DB_VERSION = 1;

function App() {
    // Track which page is currently visible
    const [currentPage, setCurrentPage] = useState('add');

    // Open the database once and reuse the same instance across renders
    const dbInstance = useMemo(() => openCostsDB(DB_NAME, DB_VERSION), []);

    // Return the correct component based on the active navigation tab
    const renderPage = () => {
        switch (currentPage) {
            case 'add':
                return <AddCostForm dbInstance={dbInstance} />;
            case 'report':
                return <MonthlyReport dbName={DB_NAME} />;
            case 'pie':
                return <PieChartView dbName={DB_NAME} />;
            case 'bar':
                return <BarChartView dbName={DB_NAME} />;
```

```
            case 'settings':
                return <Settings />;
            default:
                return <AddCostForm dbInstance={dbInstance} />;
        }
    };

    return (
        <Box sx={{ flexGrow: 1, minHeight: '100vh' }}>
            {/* Navigation bar is always visible at the top */}
            <NavBar currentPage={currentPage} onNavigate={setCurrentPage} />
            <Container maxWidth='md' sx={{ mt: 4, pb: 4 }}>
                {renderPage()}
            </Container>
        </Box>
    );
}

export default App;
```

src/App.css

```
.counter {
  font-size: 16px;
  padding: 5px 10px;
  border-radius: 5px;
  color: var(--accent);
  background: var(--accent-bg);
  border: 2px solid transparent;
  transition: border-color 0.3s;
  margin-bottom: 24px;

  &:hover {
    border-color: var(--accent-border);
  }
  &:focus-visible {
    outline: 2px solid var(--accent);
    outline-offset: 2px;
  }
}

.hero {
  position: relative;

  .base,
  .framework,
  .vite {
    inset-inline: 0;
    margin: 0 auto;
  }

  .base {
    width: 170px;
    position: relative;
    z-index: 0;
  }

  .framework,
```

```
  .vite {
    position: absolute;
  }

  .framework {
    z-index: 1;
    top: 34px;
    height: 28px;
    transform: perspective(2000px) rotateZ(300deg) rotateX(44deg) rotateY(39deg)
      scale(1.4);
  }

  .vite {
    z-index: 0;
    top: 107px;
    height: 26px;
    width: auto;
    transform: perspective(2000px) rotateZ(300deg) rotateX(40deg) rotateY(39deg)
      scale(0.8);
  }
}

#center {
  display: flex;
  flex-direction: column;
  gap: 25px;
  place-content: center;
  place-items: center;
  flex-grow: 1;

  @media (max-width: 1024px) {
    padding: 32px 20px 24px;
    gap: 18px;
  }
}

#next-steps {
  display: flex;
  border-top: 1px solid var(--border);
```

```
  text-align: left;

  & > div {
    flex: 1 1 0;
    padding: 32px;
    @media (max-width: 1024px) {
      padding: 24px 20px;
    }
  }

  .icon {
    margin-bottom: 16px;
    width: 22px;
    height: 22px;
  }

  @media (max-width: 1024px) {
    flex-direction: column;
    text-align: center;
  }
}

#docs {
  border-right: 1px solid var(--border);

  @media (max-width: 1024px) {
    border-right: none;
    border-bottom: 1px solid var(--border);
  }
}

#next-steps ul {
  list-style: none;
  padding: 0;
  display: flex;
  gap: 8px;
  margin: 32px 0 0;

  .logo {
```

```
    height: 18px;
  }

  a {
    color: var(--text-h);
    font-size: 16px;
    border-radius: 6px;
    background: var(--social-bg);
    display: flex;
    padding: 6px 12px;
    align-items: center;
    gap: 8px;
    text-decoration: none;
    transition: box-shadow 0.3s;

    &:hover {
      box-shadow: var(--shadow);
    }
    .button-icon {
      height: 18px;
      width: 18px;
    }
  }

  @media (max-width: 1024px) {
    margin-top: 20px;
    flex-wrap: wrap;
    justify-content: center;

    li {
      flex: 1 1 calc(50% - 8px);
    }

    a {
      width: 100%;
      justify-content: center;
      box-sizing: border-box;
    }
  }
```

```
}

#spacer {
  height: 88px;
  border-top: 1px solid var(--border);
  @media (max-width: 1024px) {
    height: 48px;
  }
}

.ticks {
  position: relative;
  width: 100%;

  &::before,
  &::after {
    content: '';
    position: absolute;
    top: -4.5px;
    border: 5px solid transparent;
  }

  &::before {
    left: 0;
    border-left-color: var(--border);
  }
  &::after {
    right: 0;
    border-right-color: var(--border);
  }
}
```

src/index.css

```
/* index.css - Global base styles. MUI handles component-level styling. */

@import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&family=Oxanium:wght@400;600;700;800&display=swap');

*,
*::before,
*::after {
    box-sizing: border-box;
}

body {
    margin: 0;
    font-family: 'Inter', 'Roboto', 'Segoe UI', sans-serif;
}
```

src/main.jsx

```jsx
// main.jsx
// Entry point for the React application.
// Sets up the MUI theme provider and mounts the App component.

import React from 'react';
import ReactDOM from 'react-dom/client';
import { ThemeProvider, createTheme } from '@mui/material/styles';
import CssBaseline from '@mui/material/CssBaseline';
import App from './App';
import './index.css';

// Define the global MUI theme ? dark glassmorphism design
const theme = createTheme({
    palette: {
        mode: 'dark',
        primary: { main: '#90caf9' },
        secondary: { main: '#ce93d8' },
        background: { default: 'transparent', paper: '#1a2540' },
        text: { primary: '#ffffff', secondary: 'rgba(255,255,255,0.7)' }
    },
    typography: {
        fontFamily: '"Inter", "Roboto", "Helvetica", "Arial", sans-serif',
        h5: { fontWeight: 700 },
        h6: { fontWeight: 600 }
    },
    components: {
        MuiCssBaseline: {
            styleOverrides: {
                body: {
                    background: 'linear-gradient(135deg, #1a1a2e 0%, #16213e 50%, #0f3460 100%)',
                    backgroundAttachment: 'fixed',
                    backgroundRepeat: 'no-repeat',
                    backgroundSize: 'cover',
                    minHeight: '100vh'
                }
            }
```

```
        },
        MuiPaper: {
            styleOverrides: {
                root: { backgroundImage: 'none', backgroundColor: '#1a2540' }
            }
        },
        MuiCard: {
            styleOverrides: {
                root: {
                    backgroundImage: 'none',
                    background: 'rgba(255, 255, 255, 0.08)',
                    backdropFilter: 'blur(20px)',
                    WebkitBackdropFilter: 'blur(20px)',
                    border: '1px solid rgba(255, 255, 255, 0.15)',
                    borderRadius: 16,
                    boxShadow: '0 8px 32px rgba(0, 0, 0, 0.37)'
                }
            }
        },
        MuiOutlinedInput: {
            styleOverrides: {
                root: {
                    background: 'rgba(255, 255, 255, 0.07)',
                    borderRadius: 8,
                    '& .MuiOutlinedInput-notchedOutline': {
                        borderColor: 'rgba(255, 255, 255, 0.25)'
                    },
                    '&:hover .MuiOutlinedInput-notchedOutline': {
                        borderColor: 'rgba(255, 255, 255, 0.5)'
                    },
                    '&.Mui-focused .MuiOutlinedInput-notchedOutline': {
                        borderColor: 'rgba(255, 255, 255, 0.8)'
                    }
                }
            }
        },
        MuiInputLabel: {
            styleOverrides: {
                root: {
```

```
                color: 'rgba(255, 255, 255, 0.6)',
                '&.Mui-focused': { color: 'rgba(255, 255, 255, 0.9)' }
            }
        }
    },
    MuiSelect: {
        styleOverrides: {
            icon: { color: 'rgba(255, 255, 255, 0.6)' }
        }
    },
    MuiButton: {
        styleOverrides: {
            contained: {
                background: 'rgba(255, 255, 255, 0.15)',
                backdropFilter: 'blur(10px)',
                WebkitBackdropFilter: 'blur(10px)',
                border: '1px solid rgba(255, 255, 255, 0.3)',
                color: '#ffffff',
                fontWeight: 600,
                letterSpacing: '0.05em',
                boxShadow: '0 4px 15px rgba(0, 0, 0, 0.2)',
                '&:hover': {
                    background: 'rgba(255, 255, 255, 0.28)',
                    boxShadow: '0 6px 20px rgba(255, 255, 255, 0.15)',
                    transform: 'translateY(-1px)'
                },
                '&.Mui-disabled': {
                    background: 'rgba(255, 255, 255, 0.05)',
                    color: 'rgba(255, 255, 255, 0.3)',
                    border: '1px solid rgba(255, 255, 255, 0.1)'
                },
                transition: 'all 0.25s ease'
            }
        }
    },
    MuiTableCell: {
        styleOverrides: {
            root: {
                borderBottom: 'rgba(255, 255, 255, 0.1)',
```

```
                    color: 'rgba(255, 255, 255, 0.87)'
                },
                head: {
                    color: 'rgba(255, 255, 255, 0.55)',
                    fontWeight: 600,
                    fontSize: '0.72rem',
                    textTransform: 'uppercase',
                    letterSpacing: '0.08em'
                }
            }
        },
        MuiTableRow: {
            styleOverrides: {
                root: {
                    '&:hover': { backgroundColor: 'rgba(255, 255, 255, 0.04)' }
                }
            }
        },
        MuiDivider: {
            styleOverrides: {
                root: { borderColor: 'rgba(255, 255, 255, 0.15)' }
            }
        },
        MuiCircularProgress: {
            styleOverrides: {
                root: { color: 'rgba(255, 255, 255, 0.8)' }
            }
        },
        MuiAlert: {
            styleOverrides: {
                root: {
                    backdropFilter: 'blur(8px)',
                    WebkitBackdropFilter: 'blur(8px)',
                    borderRadius: 8,
                    border: '1px solid rgba(255, 255, 255, 0.1)'
                }
            }
        }
    }
```

```
});

ReactDOM.createRoot(document.getElementById('root')).render(
    <React.StrictMode>
        <ThemeProvider theme={theme}>
            {/* CssBaseline normalizes browser default styles across all browsers */}
            <CssBaseline />
            <App />
        </ThemeProvider>
    </React.StrictMode>
);
```

src/db.js

```
// src/db.js
// ES module version of the Cost Manager database library.
// Used by React components via import statements.

// Default exchange rates relative to USD (e.g. ILS:3.4 means 3.4 ILS = 1 USD)
const DEFAULT_RATES = { USD: 1, GBP: 0.6, EURO: 0.7, ILS: 3.4 };

// localStorage keys for costs, cached rates, and the settings URL
const RATES_KEY = 'cm_exchange_rates';
const RATES_URL_KEY = 'cm_rates_url';

// The default URL for fetching exchange rates (static JSON hosted by the team)
const DEFAULT_RATES_URL = 'https://eliyazakay.github.io/costmanager-rates/rates.json';

/*
 * Fetches the latest exchange rates from the configured URL and caches them
 * in localStorage so that getReport can use them synchronously.
 * Falls back to default rates if the fetch fails for any reason.
 */
async function fetchAndCacheRates() {
    // Read the user-configured URL from settings, or use the default
    const url = localStorage.getItem(RATES_URL_KEY) || DEFAULT_RATES_URL;

    try {
        const response = await fetch(url);
        // Throw an error if the server returned a non-OK status
        if (!response.ok) {
            throw new Error(`Rates fetch failed with status ${response.status}`);
        }
        const rates = await response.json();
        // Persist the fetched rates so other functions can use them
        localStorage.setItem(RATES_KEY, JSON.stringify(rates));
        return rates;
    } catch (error) {
        // On failure, return whatever is cached or the built-in defaults
        console.error('Could not fetch exchange rates:', error);
```

```
        const cached = localStorage.getItem(RATES_KEY);
        return cached ? JSON.parse(cached) : DEFAULT_RATES;
    }
}

// Returns the most recently cached exchange rates, or the built-in defaults
function getCachedRates() {
    const stored = localStorage.getItem(RATES_KEY);
    if (stored) {
        return JSON.parse(stored);
    }
    return DEFAULT_RATES;
}

/*
 * Converts a monetary amount from one currency to another.
 * All conversions route through USD as the common base currency.
 * Rate meaning: rates[currency] units of that currency equal 1 USD.
 */
function convertCurrency(amount, fromCurrency, toCurrency, rates) {
    // Convert the source amount to USD, then to the target currency
    const amountInUSD = amount / rates[fromCurrency];
    return amountInUSD * rates[toCurrency];
}

/*
 * Opens or initializes a named cost database stored in localStorage.
 * Returns a database object exposing the addCost method.
 * Kicks off an async fetch of the latest exchange rates in the background.
 */
function openCostsDB(databaseName, databaseVersion) {
    // Initialize with an empty array if this database does not exist yet
    if (!localStorage.getItem(databaseName)) {
        localStorage.setItem(databaseName, JSON.stringify([]));
    }

    // Refresh exchange rates in the background without blocking the caller
    fetchAndCacheRates();

```

```
    // Return the database object bound to this specific database name
    return {
        /*
         * Adds a new cost item and persists it in localStorage.
         * The current date is automatically attached to the item.
         */
        addCost: function (cost) {
            const costs = JSON.parse(localStorage.getItem(databaseName) || '[]');
            const now = new Date();

            // Build the complete cost object including today's date
            const newCost = {
                sum: cost.sum,
                currency: cost.currency,
                category: cost.category,
                description: cost.description,
                date: {
                    day: now.getDate(),
                    month: now.getMonth() + 1,
                    year: now.getFullYear()
                }
            };

            // Save the updated costs array back to localStorage
            costs.push(newCost);
            localStorage.setItem(databaseName, JSON.stringify(costs));
            return newCost;
        }
    };
}

/*
 * Generates a cost report for a specific month, year, and display currency.
 * Fetches the latest exchange rates from the server before calculating totals.
 * Defaults to the current month and year when those arguments are omitted.
 */
async function getReport(dbName, currency, year, month) {
    const now = new Date();
    // Use current year and month as defaults when not explicitly provided
```

```
    const targetYear = (year !== undefined) ? year : now.getFullYear();
    const targetMonth = (month !== undefined) ? month : (now.getMonth() + 1);

    // Fetch fresh rates from the server before building the report
    const rates = await fetchAndCacheRates();

    // Load the full cost list from localStorage
    const allCosts = JSON.parse(localStorage.getItem(dbName) || '[]');

    // Keep only costs that belong to the requested month and year
    const filteredCosts = allCosts.filter(
        cost => cost.date.year === targetYear && cost.date.month === targetMonth
    );

    // Sum all costs after converting each one to the requested currency
    let totalSum = 0;
    filteredCosts.forEach(cost => {
        totalSum += convertCurrency(cost.sum, cost.currency, currency, rates);
    });

    // Round the total to two decimal places
    totalSum = Math.round(totalSum * 100) / 100;

    return {
        year: targetYear,
        month: targetMonth,
        costs: filteredCosts,
        total: { currency: currency, sum: totalSum }
    };
}

/*
 * Generates a yearly summary showing total costs per month.
 * Used by the Bar Chart feature.
 */
async function getYearlySummary(dbName, currency, year) {
    const targetYear = year !== undefined ? year : new Date().getFullYear();

    // Fetch fresh exchange rates before calculating
```

```
    const rates = await fetchAndCacheRates();
    const allCosts = JSON.parse(localStorage.getItem(dbName) || '[]');

    // Build a 12-entry array, one total per month
    const monthlyTotals = Array.from({ length: 12 }, (_, i) => ({
        month: i + 1,
        total: 0
    }));

    // Accumulate costs into the correct month bucket
    allCosts.forEach(cost => {
        if (cost.date.year === targetYear) {
            const index = cost.date.month - 1;
            monthlyTotals[index].total += convertCurrency(
                cost.sum, cost.currency, currency, rates
            );
        }
    });

    // Round each monthly total to two decimal places
    monthlyTotals.forEach(entry => {
        entry.total = Math.round(entry.total * 100) / 100;
    });

    return { year: targetYear, currency: currency, months: monthlyTotals };
}

/*
 * Saves a custom exchange rate URL to localStorage.
 * This URL is used by all subsequent calls to fetchAndCacheRates.
 */
function saveRatesUrl(url) {
    localStorage.setItem(RATES_URL_KEY, url);
}

// Returns the currently saved exchange rate URL, or the default
function getRatesUrl() {
    return localStorage.getItem(RATES_URL_KEY) || DEFAULT_RATES_URL;
}
```

```
193
194 export {
195     openCostsDB,
196     getReport,
197     getYearlySummary,
198     fetchAndCacheRates,
199     saveRatesUrl,
200     getRatesUrl,
201     getCachedRates
202 };
```

db.js (root - vanilla version)

```
// db.js
// Vanilla JavaScript library for the Cost Manager application.
// Attaches the db object to the global (window) scope via <script> tag.

/*
 * Module pattern using an IIFE to encapsulate all internal state
 * while exposing only the public API through the returned object.
 * This avoids polluting the global scope beyond the single db property.
 */
window.db = (function () {

    // Tracks the currently active database name set by openCostsDB
    let activeDBName = null;

    // Default exchange rates relative to USD (e.g. ILS:3.4 means 3.4 ILS = 1 USD)
    const DEFAULT_RATES = { USD: 1, GBP: 0.6, EURO: 0.7, ILS: 3.4 };

    // localStorage key used to cache the most recently fetched exchange rates
    const RATES_KEY = 'cm_exchange_rates';

    // Retrieves cached exchange rates from localStorage, falling back to defaults
    function getExchangeRates() {
        const stored = localStorage.getItem(RATES_KEY);
        // Return parsed rates if previously cached, otherwise use built-in defaults
        if (stored) {
            return JSON.parse(stored);
        }
        return DEFAULT_RATES;
    }

    /*
     * Converts a monetary amount from one currency to another.
     * All conversions route through USD as the common base currency.
     * Rate meaning: rate[currency] units of that currency equal 1 USD.
     */
    function convertCurrency(amount, fromCurrency, toCurrency) {
```

```
        const rates = getExchangeRates();
        // Convert source amount to USD first, then to the target currency
        const amountInUSD = amount / rates[fromCurrency];
        return amountInUSD * rates[toCurrency];
    }

    /*
     * Opens or initializes a named cost database stored in localStorage.
     * Returns a database object exposing the addCost method.
     */
    function openCostsDB(databaseName, databaseVersion) {
        // Store the active DB name so getReport can access the same data
        activeDBName = databaseName;

        // Initialize with an empty array if this database has not been created yet
        if (!localStorage.getItem(databaseName)) {
            localStorage.setItem(databaseName, JSON.stringify([]));
        }

        // Return the database object with the addCost method attached
        return {
            /*
             * Adds a new cost item to the active database.
             * The current date is automatically attached to the item.
             */
            addCost: function (cost) {
                const costs = JSON.parse(localStorage.getItem(activeDBName) || '[]');
                const now = new Date();

                // Build the complete cost object including today's date
                const newCost = {
                    sum: cost.sum,
                    currency: cost.currency,
                    category: cost.category,
                    description: cost.description,
                    date: {
                        day: now.getDate(),
                        month: now.getMonth() + 1,
                        year: now.getFullYear()
```

```
                    }
                };

                // Persist the updated costs array back to localStorage
                costs.push(newCost);
                localStorage.setItem(activeDBName, JSON.stringify(costs));
                return newCost;
            }
        };
    }

    /*
     * Generates a cost report for a specific month, year, and display currency.
     * Defaults to the current month and year when those arguments are omitted.
     * Uses cached exchange rates from localStorage for all currency conversions.
     */
    function getReport(currency, year, month) {
        if (!activeDBName) {
            throw new Error('No database is open. Call openCostsDB first.');
        }

        const now = new Date();
        // Fall back to current year and month if not explicitly provided
        const targetYear = (year !== undefined) ? year : now.getFullYear();
        const targetMonth = (month !== undefined) ? month : (now.getMonth() + 1);

        // Load the full cost list from localStorage
        const allCosts = JSON.parse(localStorage.getItem(activeDBName) || '[]');

        // Keep only the costs that belong to the requested month and year
        const filteredCosts = allCosts.filter(
            cost => cost.date.year === targetYear && cost.date.month === targetMonth
        );

        // Accumulate the total by converting each cost to the requested currency
        let totalSum = 0;
        filteredCosts.forEach(cost => {
            totalSum += convertCurrency(cost.sum, cost.currency, currency);
        });
```

```
115
116         // Round the total to two decimal places for clean output
117         totalSum = Math.round(totalSum * 100) / 100;
118
119         return {
120             year: targetYear,
121             month: targetMonth,
122             costs: filteredCosts,
123             total: { currency: currency, sum: totalSum }
124         };
125     }
126
127     // Expose only the public API of the db module
128     return {
129         openCostsDB: openCostsDB,
130         getReport: getReport
131     };
132
133 }());
```

index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>costmanager</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

vite.config.js

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
})
```

package.json

```
{
  "name": "costmanager",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "@emotion/react": "^11.14.0",
    "@emotion/styled": "^11.14.1",
    "@mui/icons-material": "^9.0.1",
    "@mui/material": "^9.0.1",
    "react": "^19.2.6",
    "react-dom": "^19.2.6",
    "recharts": "^3.8.1"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "@types/react": "^19.2.14",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^6.0.1",
    "eslint": "^10.3.0",
    "eslint-plugin-react-hooks": "^7.1.1",
    "eslint-plugin-react-refresh": "^0.5.2",
    "globals": "^17.6.0",
    "vite": "^8.0.12"
  }
}
```

rates.json

```
{"USD":1,"GBP":0.6,"EURO":0.7,"ILS":3.4}
```

src/components/NavBar.jsx

```
// NavBar.jsx
// Top navigation bar for the Cost Manager application.
// Renders the app title and tab-based navigation links.

import { AppBar, Toolbar, Tabs, Tab, Typography } from '@mui/material';

// Navigation tab definitions with display labels and route keys
const NAV_TABS = [
    { label: 'Add Cost', value: 'add' },
    { label: 'Monthly Report', value: 'report' },
    { label: 'Pie Chart', value: 'pie' },
    { label: 'Bar Chart', value: 'bar' },
    { label: 'Settings', value: 'settings' }
];

/*
 * NavBar renders the application's top AppBar with a tab set for navigation.
 * It receives the current page key and a callback to change the active page.
 */
function NavBar({ currentPage, onNavigate }) {
    // Handle tab change events and propagate the new page key upward
    const handleChange = (event, newValue) => {
        onNavigate(newValue);
    };

    return (
        <AppBar
            position='static'
            elevation={0}
            sx={{
                background: 'rgba(255, 255, 255, 0.08)',
                backdropFilter: 'blur(12px)',
                WebkitBackdropFilter: 'blur(12px)',
                borderBottom: '1px solid rgba(255, 255, 255, 0.15)',
                boxShadow: 'none'
            }}
```

```
        >
            <Toolbar sx={{ gap: 1 }}>
                <Typography
                    sx={{
                        mr: 3,
                        fontFamily: '"Oxanium", "Inter", sans-serif',
                        fontWeight: 700,
                        fontSize: '1.75rem',
                        letterSpacing: '0.06em',
                        color: '#fff',
                        userSelect: 'none',
                        textTransform: 'uppercase',
                        textShadow: `
                            0 0 6px rgba(255, 255, 255, 1),
                            0 0 14px rgba(255, 255, 255, 0.85),
                            0 0 28px rgba(255, 255, 255, 0.65),
                            0 0 55px rgba(255, 255, 255, 0.35)
                        `
                    }}
                >
                    Spendly
                </Typography>

                {/* Navigation tabs aligned to the right of the title */}
                <Tabs
                    value={currentPage}
                    onChange={handleChange}
                    textColor='inherit'
                    TabIndicatorProps={{
                        style: { backgroundColor: '#90caf9', height: 3, borderRadius: '3px 3px 0 0' }
                    }}
                    variant='scrollable'
                    scrollButtons='auto'
                >
                    {NAV_TABS.map(tab => (
                        <Tab
                            key={tab.value}
                            label={tab.label}
                            value={tab.value}
```

```
                                sx={{
                                    letterSpacing: '0.04em',
                                    fontWeight: 500,
                                    opacity: 0.72,
                                    px: 2,
                                    '&.Mui-selected': { opacity: 1, fontWeight: 600 }
                                }}
                            />
                        ))}
                    </Tabs>
                </Toolbar>
            </AppBar>
        );
}

export default NavBar;
```

src/components/AddCostForm.jsx

```jsx
// AddCostForm.jsx
// Form component that allows the user to add a new cost item.
// On submit the cost is persisted to localStorage via the db module.

import { useState } from 'react';
import {
    Box, Button, Card, CardContent, FormControl,
    InputLabel, MenuItem, Select, TextField, Typography, Alert
} from '@mui/material';

// Supported currencies as required by the project specification
const CURRENCIES = ['USD', 'ILS', 'GBP', 'EURO'];

// Available cost categories for the user to choose from
const CATEGORIES = [
    'Food', 'Education', 'Health', 'Entertainment',
    'Transportation', 'Shopping', 'Utilities', 'Other'
];

/*
 * AddCostForm renders a card with input fields for sum, currency,
 * category, and description. On submit it calls dbInstance.addCost
 * and shows a success or error message to the user.
 */
function AddCostForm({ dbInstance }) {
    // Form field state
    const [sum, setSum] = useState('');
    const [currency, setCurrency] = useState('USD');
    const [category, setCategory] = useState('');
    const [description, setDescription] = useState('');

    // Feedback message shown after a submit attempt
    const [message, setMessage] = useState(null);

    // Resets all form fields to their initial empty state
    const resetForm = () => {
```

```
        setSum('');
        setCurrency('USD');
        setCategory('');
        setDescription('');
    };

    // Validates inputs and adds the cost item to the database
    const handleSubmit = (event) => {
        event.preventDefault();
        setMessage(null);

        // Basic validation: sum must be a positive number
        const parsedSum = Number(sum);
        if (!sum || parsedSum <= 0 || isNaN(parsedSum)) {
            setMessage({ type: 'error', text: 'Please enter a valid positive sum.' });
            return;
        }

        // Category selection is required
        if (!category) {
            setMessage({ type: 'error', text: 'Please select a category.' });
            return;
        }

        // Description must not be empty
        if (!description.trim()) {
            setMessage({ type: 'error', text: 'Please enter a description.' });
            return;
        }

        // Persist the new cost item using the db module
        const result = dbInstance.addCost({
            sum: parsedSum,
            currency: currency,
            category: category,
            description: description.trim()
        });

        // Show success feedback and reset the form
```

```
        if (result) {
            setMessage({ type: 'success', text: 'Cost item added successfully!' });
            resetForm();
        }
    };

    return (
        <Card elevation={0}>
            <CardContent sx={{ p: 4, '&:last-child': { pb: 4 } }}>
                <Typography variant='h5' gutterBottom>
                    Add New Cost
                </Typography>

                {/* Show feedback alert when a message exists */}
                {message && (
                    <Alert severity={message.type} sx={{ mb: 2 }}>
                        {message.text}
                    </Alert>
                )}

                <Box component='form' onSubmit={handleSubmit} noValidate>
                    {/* Sum and Currency on the same row */}
                    <Box sx={{ display: 'flex', gap: 2, mb: 2 }}>
                        <TextField
                            label='Sum'
                            type='number'
                            value={sum}
                            onChange={e => setSum(e.target.value)}
                            required
                            fullWidth
                            inputProps={{ min: 0, step: '0.01' }}
                        />

                        {/* Currency selector */}
                        <FormControl fullWidth required>
                            <InputLabel>Currency</InputLabel>
                            <Select
                                value={currency}
                                label='Currency'
```

```
                            onChange={e => setCurrency(e.target.value)}
                        >
                            {CURRENCIES.map(c => (
                                <MenuItem key={c} value={c}>{c}</MenuItem>
                            ))}
                        </Select>
                    </FormControl>
                </Box>

                {/* Category selector */}
                <FormControl fullWidth required sx={{ mb: 2 }}>
                    <InputLabel>Category</InputLabel>
                    <Select
                        value={category}
                        label='Category'
                        onChange={e => setCategory(e.target.value)}
                    >
                        {CATEGORIES.map(cat => (
                            <MenuItem key={cat} value={cat}>{cat}</MenuItem>
                        ))}
                    </Select>
                </FormControl>

                {/* Description text field */}
                <TextField
                    label='Description'
                    value={description}
                    onChange={e => setDescription(e.target.value)}
                    required
                    fullWidth
                    multiline
                    rows={2}
                    sx={{ mb: 3 }}
                />

                <Button
                    type='submit'
                    variant='contained'
                    size='large'
```

```
                        fullWidth
                    >
                        Add Cost
                    </Button>
                </Box>
            </CardContent>
        </Card>
    );
}

export default AddCostForm;
```

src/components/MonthlyReport.jsx

```
// MonthlyReport.jsx
// Displays a detailed cost report for a user-selected month, year, and currency.
// Fetches fresh exchange rates before calculating the total.

import { useState } from 'react';
import {
    Box, Button, Card, CardContent, CircularProgress, FormControl,
    InputLabel, MenuItem, Select, Table, TableBody, TableCell,
    TableHead, TableRow, Typography, Alert, Divider
} from '@mui/material';
import { getReport } from '../db';

// Supported display currencies
const CURRENCIES = ['USD', 'ILS', 'GBP', 'EURO'];

// Month names used to build the month selector
const MONTH_NAMES = [
    'January', 'February', 'March', 'April', 'May', 'June',
    'July', 'August', 'September', 'October', 'November', 'December'
];

// Build a list of recent years for the year selector (current year ± 4)
const buildYearOptions = () => {
    const currentYear = new Date().getFullYear();
    return Array.from({ length: 5 }, (_, i) => currentYear - i);
};

/*
 * MonthlyReport allows the user to select a month, year, and display currency,
 * then fetches and displays all costs for that period along with the total.
 */
function MonthlyReport({ dbName }) {
    const now = new Date();

    // Selector state ? default to the current month, year, and USD
    const [month, setMonth] = useState(now.getMonth() + 1);
```

```
    const [year, setYear] = useState(now.getFullYear());
    const [currency, setCurrency] = useState('USD');

    // Report data and UI state
    const [reportData, setReportData] = useState(null);
    const [loading, setLoading] = useState(false);
    const [error, setError] = useState(null);

    const yearOptions = buildYearOptions();

    // Fetches the report from the db module and updates state
    const handleGetReport = async () => {
        setLoading(true);
        setError(null);
        setReportData(null);

        try {
            const data = await getReport(dbName, currency, year, month);
            setReportData(data);
        } catch (err) {
            setError('Failed to load report. Please try again.');
            console.error(err);
        } finally {
            setLoading(false);
        }
    };

    return (
        <Card elevation={0}>
            <CardContent sx={{ p: 4, '&:last-child': { pb: 4 } }}>
                <Typography variant='h5' gutterBottom>
                    Monthly Report
                </Typography>

                {/* Selectors row: month, year, and display currency */}
                <Box sx={{ display: 'flex', gap: 2, mb: 3, flexWrap: 'wrap' }}>
                    <FormControl sx={{ minWidth: 150 }}>
                        <InputLabel>Month</InputLabel>
                        <Select
```

```
                        value={month}
                        label='Month'
                        onChange={e => setMonth(Number(e.target.value))}
                    >
                        {MONTH_NAMES.map((name, index) => (
                            <MenuItem key={name} value={index + 1}>{name}</MenuItem>
                        ))}
                    </Select>
                </FormControl>

                {/* Year selector */}
                <FormControl sx={{ minWidth: 120 }}>
                    <InputLabel>Year</InputLabel>
                    <Select
                        value={year}
                        label='Year'
                        onChange={e => setYear(Number(e.target.value))}
                    >
                        {yearOptions.map(y => (
                            <MenuItem key={y} value={y}>{y}</MenuItem>
                        ))}
                    </Select>
                </FormControl>

                {/* Display currency selector */}
                <FormControl sx={{ minWidth: 120 }}>
                    <InputLabel>Currency</InputLabel>
                    <Select
                        value={currency}
                        label='Currency'
                        onChange={e => setCurrency(e.target.value)}
                    >
                        {CURRENCIES.map(c => (
                            <MenuItem key={c} value={c}>{c}</MenuItem>
                        ))}
                    </Select>
                </FormControl>

                <Button
```

```
                        variant='contained'
                        onClick={handleGetReport}
                        disabled={loading}
                        sx={{ height: 56, whiteSpace: 'nowrap' }}
                    >
                        Get Report
                    </Button>
                </Box>

                {/* Loading spinner shown while fetching */}
                {loading && (
                    <Box sx={{ display: 'flex', justifyContent: 'center', mt: 3 }}>
                        <CircularProgress />
                    </Box>
                )}

                {/* Error alert shown on fetch failure */}
                {error && <Alert severity='error'>{error}</Alert>}

                {/* Report results table */}
                {reportData && (
                    <Box>
                        <Divider sx={{ mb: 2 }} />

                        {reportData.costs.length === 0 ? (
                            <Typography color='text.secondary'>
                                No costs found for {MONTH_NAMES[month - 1]} {year}.
                            </Typography>
                        ) : (
                            <Table size='small'>
                                <TableHead>
                                    <TableRow>
                                        <TableCell><strong>Day</strong></TableCell>
                                        <TableCell><strong>Category</strong></TableCell>
                                        <TableCell><strong>Description</strong></TableCell>
                                        <TableCell align='right'><strong>Sum</strong></TableCell>
                                        <TableCell><strong>Currency</strong></TableCell>
                                    </TableRow>
                                </TableHead>
```

```
                                <TableBody>
                                    {/* Render one row per cost item */}
                                    {reportData.costs.map((cost, index) => (
                                        <TableRow key={index}>
                                            <TableCell>{cost.date.day}</TableCell>
                                            <TableCell>{cost.category}</TableCell>
                                            <TableCell>{cost.description}</TableCell>
                                            <TableCell align='right'>{cost.sum}</TableCell>
                                            <TableCell>{cost.currency}</TableCell>
                                        </TableRow>
                                    ))}
                                </TableBody>
                            </Table>
                        )}

                        {/* Total row displayed below the table */}
                        <Box sx={{ mt: 2, p: 2, background: 'rgba(144, 202, 249, 0.15)', border: '1px solid rgba(144, 202, 249, 0.35)', borderRadius: 2 }}>
                            <Typography variant='h6'>
                                Total: {reportData.total.sum} {reportData.total.currency}
                            </Typography>
                        </Box>
                    </Box>
                )}
            </CardContent>
        </Card>
    );
}

export default MonthlyReport;
```

src/components/BarChartView.jsx

```
// BarChartView.jsx
// Displays a bar chart of total monthly costs for a user-selected year and currency.
// Each bar represents the total spending for one of the twelve months.

import { useState } from 'react';
import {
    Box, Button, Card, CardContent, CircularProgress,
    FormControl, InputLabel, MenuItem, Select, Typography, Alert
} from '@mui/material';
import BarChartIcon from '@mui/icons-material/BarChart';
import {
    BarChart, Bar, XAxis, YAxis, CartesianGrid,
    Tooltip, ResponsiveContainer, Cell
} from 'recharts';
import { getYearlySummary } from '../db';

// Supported display currencies
const CURRENCIES = ['USD', 'ILS', 'GBP', 'EURO'];

// Short month labels for the X-axis of the bar chart
const MONTH_LABELS = [
    'Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
    'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'
];

const BAR_COLOR = '#64b5f6';

// Build a list of recent years for the year selector
const buildYearOptions = () => {
    const currentYear = new Date().getFullYear();
    return Array.from({ length: 5 }, (_, i) => currentYear - i);
};

/*
 * BarChartView lets the user pick a year and currency, then renders a
 * Recharts BarChart showing the total spending per month for that year.
```

```
 */
function BarChartView({ dbName }) {
    const now = new Date();

    // Selector state ? default to the current year and USD
    const [year, setYear] = useState(now.getFullYear());
    const [currency, setCurrency] = useState('USD');

    // Chart data and UI state
    const [chartData, setChartData] = useState(null);
    const [loading, setLoading] = useState(false);
    const [error, setError] = useState(null);

    const yearOptions = buildYearOptions();

    // Fetches the yearly summary and prepares data for Recharts
    const handleGetChart = async () => {
        setLoading(true);
        setError(null);
        setChartData(null);

        try {
            const summary = await getYearlySummary(dbName, currency, year);

            // Map the monthly totals to the format Recharts expects
            const data = summary.months.map((entry, index) => ({
                month: MONTH_LABELS[index],
                total: entry.total
            }));

            setChartData(data);
        } catch (err) {
            setError('Failed to load chart data. Please try again.');
            console.error(err);
        } finally {
            setLoading(false);
        }
    };

```

```
    return (
        <Card elevation={0}>
            <CardContent sx={{ p: 4, '&:last-child': { pb: 4 } }}>
                <Box sx={{ display: 'flex', alignItems: 'center', gap: 1, mb: 2 }}>
                    <BarChartIcon color='primary' sx={{ fontSize: 28 }} />
                    <Typography variant='h5'>
                        Monthly Totals - Bar Chart
                    </Typography>
                </Box>

                {/* Selectors row: year and currency */}
                <Box sx={{ display: 'flex', gap: 2, mb: 3, flexWrap: 'wrap' }}>
                    <FormControl sx={{ minWidth: 120 }}>
                        <InputLabel>Year</InputLabel>
                        <Select
                            value={year}
                            label='Year'
                            onChange={e => setYear(Number(e.target.value))}
                        >
                            {yearOptions.map(y => (
                                <MenuItem key={y} value={y}>{y}</MenuItem>
                            ))}
                        </Select>
                    </FormControl>

                    {/* Currency selector for converting all totals */}
                    <FormControl sx={{ minWidth: 120 }}>
                        <InputLabel>Currency</InputLabel>
                        <Select
                            value={currency}
                            label='Currency'
                            onChange={e => setCurrency(e.target.value)}
                        >
                            {CURRENCIES.map(c => (
                                <MenuItem key={c} value={c}>{c}</MenuItem>
                            ))}
                        </Select>
                    </FormControl>

```

```
                    <Button
                        variant='contained'
                        onClick={handleGetChart}
                        disabled={loading}
                        sx={{ height: 56, whiteSpace: 'nowrap' }}
                    >
                        Show Chart
                    </Button>
                </Box>

                {/* Loading indicator */}
                {loading && (
                    <Box sx={{ display: 'flex', justifyContent: 'center', mt: 3 }}>
                        <CircularProgress />
                    </Box>
                )}

                {/* Error feedback */}
                {error && <Alert severity='error'>{error}</Alert>}

                {/* Bar chart rendered once data is ready */}
                {chartData && (
                    <ResponsiveContainer width='100%' height={400}>
                        <BarChart
                            data={chartData}
                            margin={{ top: 10, right: 30, left: 10, bottom: 5 }}
                        >
                            <CartesianGrid strokeDasharray='3 3' stroke='rgba(255,255,255,0.08)' />
                            <XAxis
                                dataKey='month'
                                tick={{ fill: 'rgba(255,255,255,0.7)', fontSize: 12 }}
                                axisLine={{ stroke: 'rgba(255,255,255,0.2)' }}
                                tickLine={{ stroke: 'rgba(255,255,255,0.2)' }}
                            />
                            <YAxis
                                tick={{ fill: 'rgba(255,255,255,0.7)', fontSize: 12 }}
                                axisLine={{ stroke: 'rgba(255,255,255,0.2)' }}
                                tickLine={{ stroke: 'rgba(255,255,255,0.2)' }}
                                label={{
```

```
                                    value: currency,
                                    angle: -90,
                                    position: 'insideLeft',
                                    offset: -5,
                                    fill: 'rgba(255,255,255,0.6)'
                                }}
                            />
                            <Tooltip
                                formatter={value => [`${value} ${currency}`, 'Total']}
                                contentStyle={{
                                    background: 'rgba(10, 20, 50, 0.92)',
                                    border: '1px solid rgba(255,255,255,0.2)',
                                    borderRadius: 8,
                                    color: 'white'
                                }}
                                itemStyle={{ color: 'rgba(255,255,255,0.87)' }}
                                labelStyle={{ color: 'rgba(255,255,255,0.6)', marginBottom: 4 }}
                            />
                            <Bar dataKey='total' name='Total'>
                                {chartData.map((_, index) => (
                                    <Cell key={`bar-${index}`} fill={BAR_COLOR} />
                                ))}
                            </Bar>
                        </BarChart>
                    </ResponsiveContainer>
                )}
            </CardContent>
        </Card>
    );
}

export default BarChartView;
```

src/components/PieChartView.jsx

```jsx
// PieChartView.jsx
// Displays a pie chart of total costs grouped by category
// for a user-selected month, year, and display currency.

import { useState } from 'react';
import {
    Box, Button, Card, CardContent, CircularProgress,
    FormControl, InputLabel, MenuItem, Select, Typography, Alert
} from '@mui/material';
import { PieChart, Pie, Cell, Tooltip, Legend, ResponsiveContainer } from 'recharts';
import { getReport } from '../db';

// Supported display currencies
const CURRENCIES = ['USD', 'ILS', 'GBP', 'EURO'];

// Month names for the month selector
const MONTH_NAMES = [
    'January', 'February', 'March', 'April', 'May', 'June',
    'July', 'August', 'September', 'October', 'November', 'December'
];

const SLICE_COLORS = [
    '#64b5f6', '#ef5350', '#66bb6a', '#ffa726',
    '#ab47bc', '#26c6da', '#ec407a', '#8d6e63'
];

// Build a list of recent years for the year selector
const buildYearOptions = () => {
    const currentYear = new Date().getFullYear();
    return Array.from({ length: 5 }, (_, i) => currentYear - i);
};

/*
 * Aggregates a flat list of cost items into totals grouped by category.
 * Returns an array of { name, value } objects ready for Recharts.
 */
```

```
function buildChartData(costs, currency, rates) {
    const totals = {};

    // Sum each cost into the correct category bucket
    costs.forEach(cost => {
        const amountInUSD = cost.sum / rates[cost.currency];
        const converted = amountInUSD * rates[currency];

        if (!totals[cost.category]) {
            totals[cost.category] = 0;
        }
        totals[cost.category] += converted;
    });

    // Convert the totals map to the array format Recharts expects
    return Object.entries(totals).map(([name, value]) => ({
        name,
        value: Math.round(value * 100) / 100
    }));
}

/*
 * PieChartView lets the user pick a month, year, and currency,
 * then renders a Recharts PieChart grouped by cost category.
 */
function PieChartView({ dbName }) {
    const now = new Date();

    // Selector state ? default to the current month and year
    const [month, setMonth] = useState(now.getMonth() + 1);
    const [year, setYear] = useState(now.getFullYear());
    const [currency, setCurrency] = useState('USD');

    // Chart data and UI state
    const [chartData, setChartData] = useState(null);
    const [loading, setLoading] = useState(false);
    const [error, setError] = useState(null);

    const yearOptions = buildYearOptions();
```

```
76
77      // Fetches the monthly report and transforms it into chart data
78      const handleGetChart = async () => {
79          setLoading(true);
80          setError(null);
81          setChartData(null);
82
83          try {
84              const reportData = await getReport(dbName, currency, year, month);
85
86              // Use cached rates (already fetched inside getReport) for conversion
87              const cached = localStorage.getItem('cm_exchange_rates');
88              const rates = cached
89                  ? JSON.parse(cached)
90                  : { USD: 1, GBP: 0.6, EURO: 0.7, ILS: 3.4 };
91
92              const data = buildChartData(reportData.costs, currency, rates);
93              setChartData(data);
94          } catch (err) {
95              setError('Failed to load chart data. Please try again.');
96              console.error(err);
97          } finally {
98              setLoading(false);
99          }
100     };
101
102     return (
103         <Card elevation={0}>
104             <CardContent sx={{ p: 4, '&:last-child': { pb: 4 } }}>
105                 <Typography variant='h5' gutterBottom>
106                     Costs by Category - Pie Chart
107                 </Typography>
108
109                 {/* Selectors row */}
110                 <Box sx={{ display: 'flex', gap: 2, mb: 3, flexWrap: 'wrap' }}>
111                     <FormControl sx={{ minWidth: 150 }}>
112                         <InputLabel>Month</InputLabel>
113                         <Select
114                             value={month}
```

```
                        label='Month'
                        onChange={e => setMonth(Number(e.target.value))}
                    >
                        {MONTH_NAMES.map((name, index) => (
                            <MenuItem key={name} value={index + 1}>{name}</MenuItem>
                        ))}
                    </Select>
                </FormControl>

                {/* Year selector */}
                <FormControl sx={{ minWidth: 120 }}>
                    <InputLabel>Year</InputLabel>
                    <Select
                        value={year}
                        label='Year'
                        onChange={e => setYear(Number(e.target.value))}
                    >
                        {yearOptions.map(y => (
                            <MenuItem key={y} value={y}>{y}</MenuItem>
                        ))}
                    </Select>
                </FormControl>

                {/* Currency selector */}
                <FormControl sx={{ minWidth: 120 }}>
                    <InputLabel>Currency</InputLabel>
                    <Select
                        value={currency}
                        label='Currency'
                        onChange={e => setCurrency(e.target.value)}
                    >
                        {CURRENCIES.map(c => (
                            <MenuItem key={c} value={c}>{c}</MenuItem>
                        ))}
                    </Select>
                </FormControl>

                <Button
                    variant='contained'
```

```
                        onClick={handleGetChart}
                        disabled={loading}
                        sx={{ height: 56, whiteSpace: 'nowrap' }}
                    >
                        Show Chart
                    </Button>
                </Box>

                {/* Loading spinner */}
                {loading && (
                    <Box sx={{ display: 'flex', justifyContent: 'center', mt: 3 }}>
                        <CircularProgress />
                    </Box>
                )}

                {/* Error message */}
                {error && <Alert severity='error'>{error}</Alert>}

                {/* Pie chart rendered when data is available */}
                {chartData && chartData.length === 0 && (
                    <Typography color='text.secondary'>
                        No costs found for {MONTH_NAMES[month - 1]} {year}.
                    </Typography>
                )}

                {chartData && chartData.length > 0 && (
                    <ResponsiveContainer width='100%' height={400}>
                        <PieChart>
                            <Pie
                                data={chartData}
                                dataKey='value'
                                nameKey='name'
                                cx='50%'
                                cy='50%'
                                outerRadius={140}
                                labelLine={{ stroke: 'rgba(255,255,255,0.4)' }}
                                label={({ name, percent, x, y }) => (
                                    <text
                                        x={x}
```

```
                                        y={y}
                                        fill='rgba(255,255,255,0.9)'
                                        textAnchor='middle'
                                        dominantBaseline='central'
                                        fontSize={12}
                                    >
                                        {`${name} ${(percent * 100).toFixed(1)}%`}
                                    </text>
                                )}
                            >
                                {chartData.map((entry, index) => (
                                    <Cell
                                        key={`cell-${index}`}
                                        fill={SLICE_COLORS[index % SLICE_COLORS.length]}
                                    />
                                ))}
                            </Pie>
                            <Tooltip
                                formatter={value => `${value} ${currency}`}
                                contentStyle={{
                                    background: 'rgba(10, 20, 50, 0.92)',
                                    border: '1px solid rgba(255,255,255,0.2)',
                                    borderRadius: 8,
                                    color: 'white'
                                }}
                                itemStyle={{ color: 'rgba(255,255,255,0.87)' }}
                            />
                            <Legend
                                formatter={(value) => (
                                    <span style={{ color: 'rgba(255,255,255,0.85)' }}>{value}</span>
                                )}
                            />
                        </PieChart>
                    </ResponsiveContainer>
                )}
            </CardContent>
        </Card>
    );
}
```

```
232
233 export default PieChartView;
```

src/components/Settings.jsx

```jsx
// Settings.jsx
// Allows the user to configure a custom URL for fetching exchange rates.
// The URL is saved to localStorage and used by all subsequent rate fetches.

import { useState, useEffect } from 'react';
import {
    Alert, Box, Button, Card, CardContent,
    TextField, Typography, Divider
} from '@mui/material';
import { saveRatesUrl, getRatesUrl, fetchAndCacheRates } from '../db';

/*
 * Settings renders a simple form where the user can enter and save a
 * custom exchange rate URL. On save the new rates are fetched immediately
 * so the rest of the app uses them without requiring a page reload.
 */
function Settings() {
    // Pre-populate the field with whatever URL is currently configured
    const [url, setUrl] = useState('');
    const [message, setMessage] = useState(null);

    // Load the current URL from the db module when the component mounts
    useEffect(() => {
        setUrl(getRatesUrl());
    }, []);

    // Saves the URL and immediately fetches fresh exchange rates
    const handleSave = async () => {
        setMessage(null);

        // Trim whitespace before saving
        const trimmedUrl = url.trim();
        if (!trimmedUrl) {
            setMessage({ type: 'error', text: 'Please enter a valid URL.' });
            return;
        }
```

```
        // Persist the new URL and refresh rates from the server
        saveRatesUrl(trimmedUrl);

        try {
            await fetchAndCacheRates();
            setMessage({ type: 'success', text: 'Settings saved and rates updated successfully!' });
        } catch (err) {
            // URL was saved even if the fetch failed ? rates will be retried later
            setMessage({ type: 'warning', text: 'URL saved, but fetching rates failed. Check the URL.' });
            console.error(err);
        }
    };

    return (
        <Card elevation={0}>
            <CardContent sx={{ p: 4, '&:last-child': { pb: 4 } }}>
                <Typography variant='h5' gutterBottom>
                    Settings
                </Typography>

                <Divider sx={{ mb: 3 }} />

                <Typography variant='subtitle1' gutterBottom>
                    Exchange Rate Source URL
                </Typography>

                <Typography variant='body2' color='text.secondary' sx={{ mb: 2 }}>
                    Enter the URL of a JSON endpoint that returns exchange rates in the
                    following format: {`{"USD":1,"GBP":0.6,"EURO":0.7,"ILS":3.4}`}
                </Typography>

                {/* Feedback alert shown after a save attempt */}
                {message && (
                    <Alert severity={message.type} sx={{ mb: 2 }}>
                        {message.text}
                    </Alert>
                )}

```

```
                {/* URL text field */}
                <Box sx={{ display: 'flex', gap: 2, alignItems: 'flex-start' }}>
                    <TextField
                        label='Exchange Rate URL'
                        value={url}
                        onChange={e => setUrl(e.target.value)}
                        fullWidth
                        placeholder='https://example.com/rates.json'
                    />

                    <Button
                        variant='contained'
                        onClick={handleSave}
                        sx={{ mt: 1, height: 56, whiteSpace: 'nowrap' }}
                    >
                        Save & Refresh
                    </Button>
                </Box>
            </CardContent>
        </Card>
    );
}

export default Settings;
```